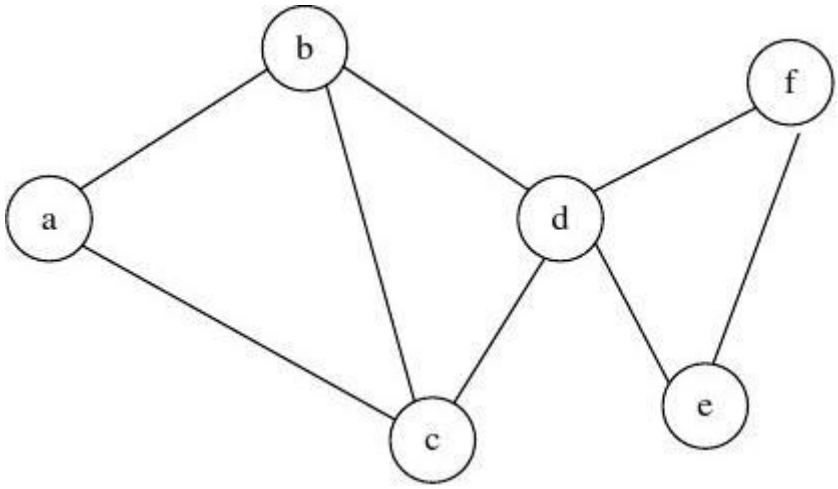
第35课-图论基础一

第35课-图论基础一
- 1 基本概念
  - 1.1 什么是图
  - 1.2 图的定义和性质
  - 1.3 图的存储
- 2 图的遍历
  - 2.1 图的深度优先遍历和广度优先遍历
- 3. 欧拉路和欧拉回路
  - 3.1 欧拉路和欧拉回路的定义
  - 3.2 打印欧拉路径
  - 3.3 是否存在欧拉路径问题
- 4. 作业
  - 4.1 骑马修栅栏　https://www.luogu.com.cn/problem/P2731
  - 4.2 欧拉路径　https://www.acwing.com/problem/content/1621/

1 基本概念

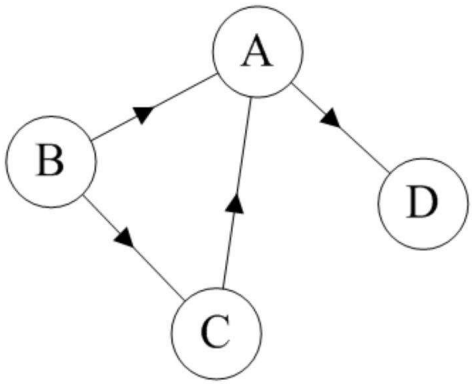
1.1 什么是图

- 图是一种数据结构，定义：$graph = (V, E)$ 。其中 V 是一个非空有限集合，代表顶点(结点)，E 代表边的集合。简单的说，点用边连起来就是图。

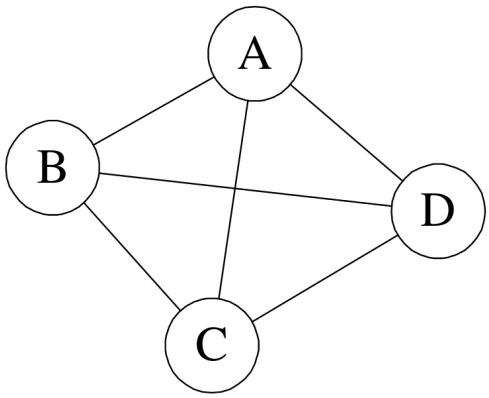


1.2 图的定义和性质

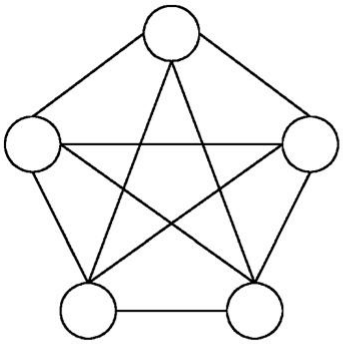
1. 有向图：图的边有方向，只能按箭头方向从一点到另一点



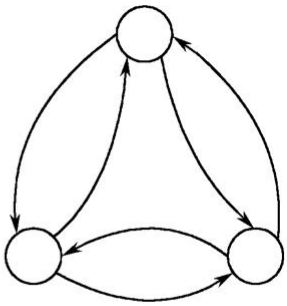
2. 无向图：图的边没有方向，可以双向通行。



3. 结点的度：无向图中与结点相连的边的数目，称为结点的度。
4. 结点的入度：在有向图中，以这个结点为终点的有向边的数目。
5. 结点的出度：在有向图中，以这个结点为起点的有向边的数目。
6. 权值：边的"费用"，可以理解为边的长度，一般情况下会是从一个点到另一个点所要花费的代价。
7. 连通：如果图中结点从 U, V 之间存在一条从 U 通过若干条边、点到达 V 的通路，则称 U, V 是连通的。
8. 回路：起点和终点相同的路径，称为回路，或"环"。负环：一个所有边的边权和为负数的环称为负环。
9. 无向完全图：无向完全图是用 n 表示图中顶点数目的一种完全图，该图中每条边都是无方向的。在无向图中，如果任意两个顶点之间都存在边，则称该图为无向完全图，无向完全图含有 $\frac{n(n-1)}{2}$ 条边，因为从每个顶点出发，和这个顶点相连都有 $n-1$ 条边，而每条边都被重复计算了一次。
10. 有向完全图：有向完全图是指图中各边都有方向，且每两个顶点之间都有两条方向相反的边连接的图。n 个顶点的有向完全图含有 $n(n-1)$ 条边。



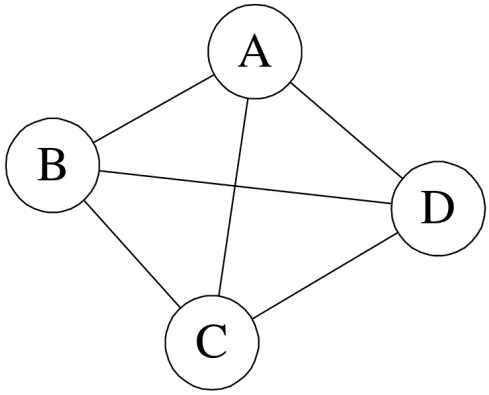
（a）无向完全图K_5



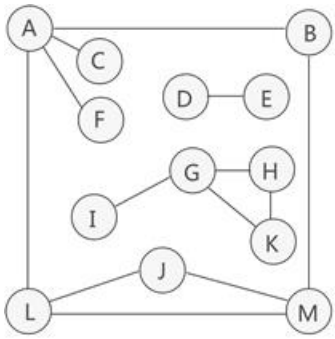
（b）有向完全图K_3

11. 稠密图：一个边数接近完全图的图。
12. 稀疏图：一个边数远远小于完全图的图。

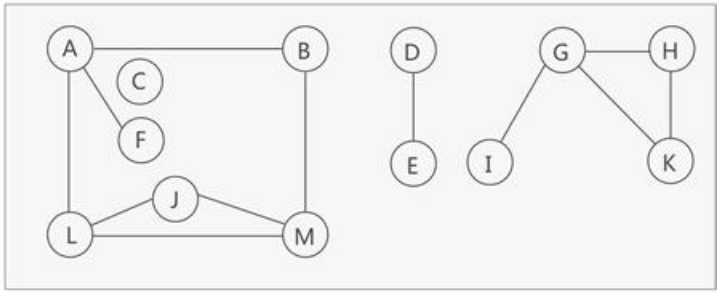
13. 奇点偶点：无向图中顶点的度若为奇数，则称这个顶点为奇点，若为偶数，则称它为偶点。
14. 定理一：无向图中所有顶点的度之和等于边数的两倍，有向图中所有顶点的入度之和等于所有顶点的出度之和。
15. 定理二：任意一个无向图一定有偶数个(或0个)奇点。
16. 无向图 G 中，如果从顶点 V 到顶点 V' 有路径（或称可连通/可通达），则称 V 和 V' 是连通的。如果对于图中任意两点 $V_i, V_j \in V$，V_i和 V_j 都是连通的，则称 G 是连通图。



17. 连通分量：(1) 非连通无向图中的极大连通子图 (2) 连通图的连通分量只有一个即本身。设$G(v,e)$是非连通图, $G'(v',e')$是连通图，如果对于 v'等于 v ,e'包含于 e,那么 G' 叫做 G 的极大连通子图



a) 非连通图



b) 连通分量

18. 强连通分量：有向图 G 中，如果对于每一对 $V_i, V_j \in V$，$V_i \neq V_j$，从 V_i 到 V_j 和从 V_j 到 V_i 都存在路径，则称 G 是强连通图。有向图中的极大强连通子图称做有向图的强连通分量。 图1不是强连通图，但它有两个强连通分量，分别是图2和单独的一个顶点D

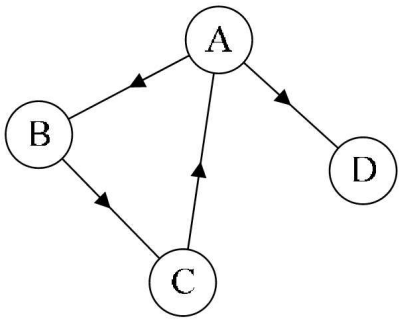


图1

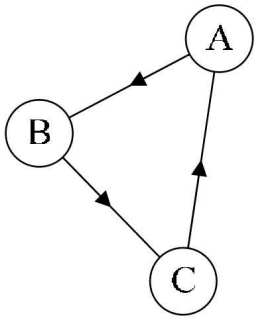
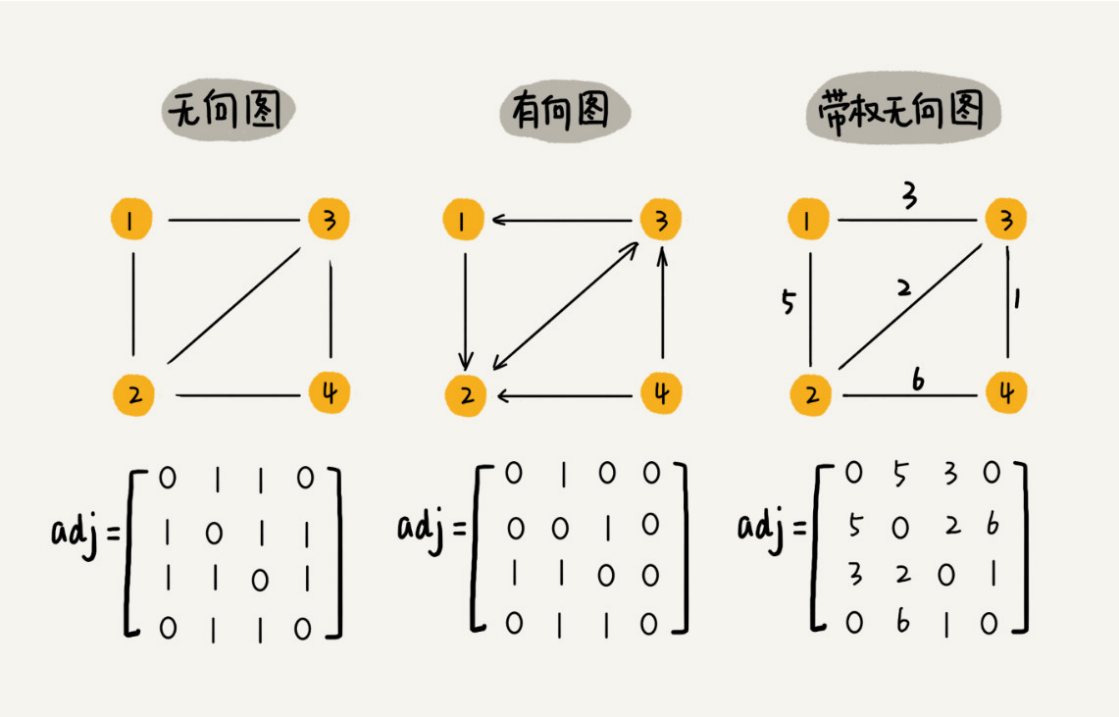


图2

19. 无向连通图最多 $n*(n-1)/2$ 条边，最少 $n-1$ 条边。如果有 $n*(n-1)/2$ 条边,即为无向完全图
20. 有向连通图最多 $n*(n-1)$ 条边，最少 n 条边。如果有 $n*(n-1)$ 条边,即为有向完全图

1.3 图的存储

1. 邻接矩阵



2. 链式前向星(数组模拟邻接表)

```
int head[N],cnt;
struct node{
    int v;
    int next;
    int w;
}edge[N];
void add_edge(int u, int v, int w){
    cnt ++;
    edge[cnt].v = v;
    edge[cnt].w = w;
    edge[cnt].next = head[u];
    head[u] = cnt;
}
int main()
{
    int u;
    cin >> u;
    for(int i = head[u];i;i=edge[i].next);
    // i=head[u],以u作为起始点的最后一条边是第i条
    // i=edge[i].next, 通过第i条边去寻找上一条以u作为起始顶点的边
    return 0;
}
```

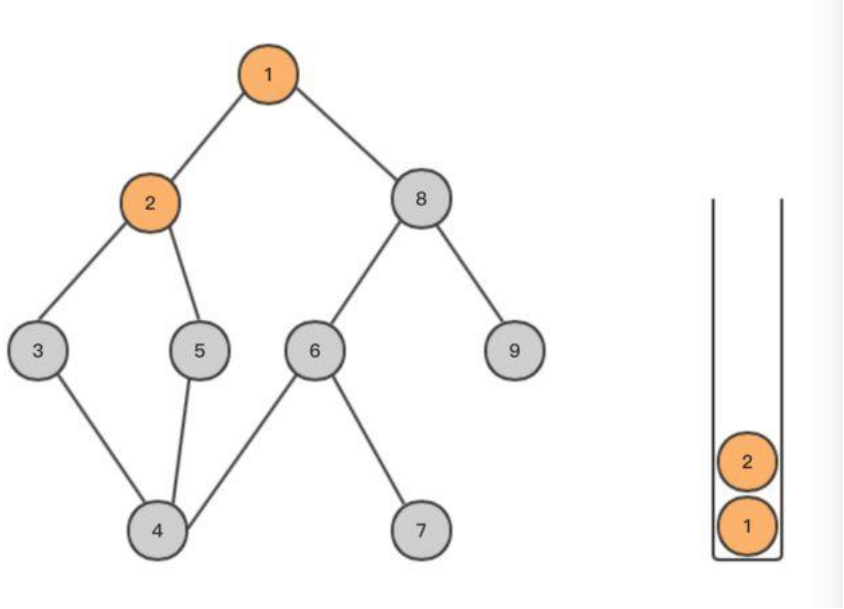
$head[u] = cnt$, 表示第 cnt 条边的起始顶点为 u, $edge[cnt].w = w$ 表示第 cnt 条边的权值是 w , $edge[cnt].v = v$, 表示第 cnt 条边的终点是 v , $edge[cnt].next = head[u]$ 表示第 cnt 条边的 $next$ 参数保存的是同样以 u 做为起始顶点的上一条边是第 $head[u]$ 条。

邻接矩存图阵容易理解，并且能在 $O(1)$ 的时间内找到任意一条边，但是缺点也非常明显，点的数量一旦增多就会非常浪费空间，在边数远远小于点数的平方时(稀疏图)，一般运用链式前向星来储存边，这样可以大大减小空间浪费，但是链式前向星不能在 $O(1)$ 时间内找到任意一条边，各有各的优点，具体的问题要有更加合适的存图策略。

2 图的遍历

2.1 图的深度优先遍历和广度优先遍历

- 从图中某一顶点出发完整地访问图中所有顶点，使每个顶点刚好被访问一次，这种操作叫做图的遍历。为了避免重复访问某个顶点，可以设置一个标志数组 visited[] ,未访问时值为 false, 访问一次改为 true。比如从A点出发，将起点标记为已经访问， visited[i] = true 。以下图为例：



- 先从1出发，1->2->3->4->5, 退回4, ->6->7, 退回6，->8->9。

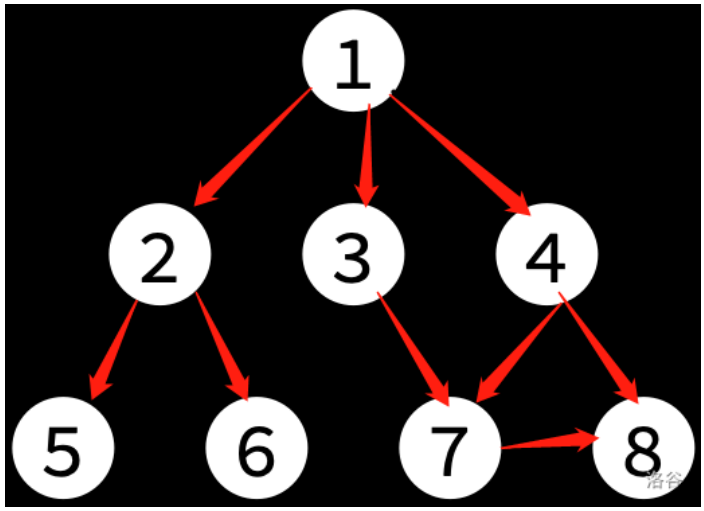
- 查找文献
- 题目描述

  时间限制:1s　　空间限制:125mb

  小K喜欢翻看洛谷博客获取知识。每篇文章可能会有若干个（也有可能没有）参考文献的链接指向别的博客文章。小K 求知欲旺盛，如果他看了某篇文章，那么他一定会去看这篇文章的参考文献（如果他之前已经看过这篇参考文献的话就不用再看它了）。

  假设洛谷博客里面一共有n(n≤10^5) 篇文章（编号为 1 到 n）以及 (m≤10^6) 条参考文献引用关系。目前小 K 已经打开了编号为 1 的一篇文章，请帮助小 K 设计一种方法，使小 K 可以不重复、不遗漏的看完所有他能看到的文章。

  这边是已经整理好的参考文献关系图，其中，文献 X → Y 表示文章 X 有参考文献 Y。不保证编号为 1 的文章没有被其他文章引用。



请对这个图分别进行 DFS 和 BFS，并输出遍历结果。如果有很多篇文章可以参阅，请先看编号较小的那篇(因此你可能需要先排序)。

输入格式

共 m+1 行，第 1 行为 2 个数，n 和 m，分别表示一共有 n(n≤10^5) 篇文章（编号为 1 到 n）以及 m(m≤10^6) 条参考文献引用关系。

接下来 m 行，每行有两个整数 X,Y 表示文章 X 有参考文献 Y。

输出格式

共 2 行。 第一行为 DFS 遍历结果，第二行为 BFS 遍历结果。

输入输出样例

输入

```
8 9
1 2
1 3
1 4
2 5
2 6
3 7
4 7
4 8
7 8
```

输出

```
1 2 5 6 3 7 8 4
1 2 3 4 5 6 7 8
```

-

```
#include <bits/stdc++.h>
#define ll long long
using namespace std;
const int maxn=1e5+10;
vector<int> G[maxn];
bool vis[maxn];
int n, m;
queue<int> q;
template<class T> inline T read(T &x){
```

```
    x = 0;T f=1;char ch=getchar();
    while(!isdigit(ch)){if(ch=='-')f=-1;ch=getchar();}
    while(isdigit(ch)){x=(x<<3)+(x<<1)+(ch^48);ch=getchar();}
    x *= f; return (T)1;
}
void dfs(int u){
    vis[u] = true;
    printf("%d ",u);
    for(int i=0;i<G[u].size();i++){
        int v = G[u][i];
        if (!vis[v]){
            dfs(v);
        }
    }
}
void bfs(int start){
    q = queue<int>();
    vis[start] = true;
    q.push(start);
    while (!q.empty()){
        int u = q.front();
        printf("%d ", u);
        q.pop();
        for(int i=0;i<G[u].size();i++){
            int v = G[u][i];
            if (!vis[v]){
                q.push(v);
                vis[v]=true;
            }
        }
    }
}
int main()
{
    read(n), read(m);
    int u, v;
    for(int i=1;i<=m;i++){
        read(u), read(v);
        G[u].push_back(v);
    }
    for(int i=1;i<=n;i++) {
        sort(G[i].begin(),G[i].end());
    }
    dfs(1);
    cout << endl;
    memset(vis, 0, sizeof(vis));
    bfs(1);
    return 0;
}
```

-

```
图的遍历
题目描述
时间限制:1s      空间限制:125mb
给出N个点，M条边的有向图，对于每个点v，求A(v)表示从点v出发，能到达的编号最大的点。

```

```
输入格式
第1 行，2 个整数N,M。
接下来M行，每行2个整数Ui,Vi ，表示边(Ui,Vi)。点用1, 2,…,N编号。

输出格式
N 个整数A(1),A(2),…,A(N)

输入输出样例
输入
4 3
1 2
2 4
4 3
输出
4 4 3 4
说明/提示
对于60% 的数据，1≤N,M≤10^3
对于100% 的数据，1≤N,M≤10^5
```

反向建边 + dfs

每次考虑每个点可以到达点编号最大的点，不如考虑较大的点可以反向到达哪些点

循环从N到1，则每个点 i 能访问到的结点的 a 值都是 i

每个点访问一次，这个a 值就是最优的，因为之后如果再访问到这个结点那么答案肯定没当前大了

```cpp
#include <bits/stdc++.h>
#define ll long long
using namespace std;
const int maxn=1e5+10;
int n, m, a[maxn];
vector<int> G[maxn];
template<class T> inline T read(T &x){
    x = 0;T f=1;char ch=getchar();
    while(!isdigit(ch)){if(ch=='-')f=-1;ch=getchar();}
    while(isdigit(ch)){x=(x<<3)+(x<<1)+(ch^48);ch=getchar();}
    x *= f; return (T)1;
}
void dfs(int u, int d){
    if (a[u]) return;
    a[u] = d;
    for(int i=0;i<G[u].size();i++){
        dfs(G[u][i], d);
    }
}
int main()
{
    read(n),read(m);
    int u, v;
    for(int i=1;i<=m;i++){
        read(u),read(v);
        G[v].push_back(u);
    }
    for(int i=n;i>=1;i--) dfs(i,i);
    for(int i=1;i<=n;i++) printf("%d ",a[i]);
    return 0;
}
```

3. 欧拉路和欧拉回路

3.1 欧拉路和欧拉回路的定义

- 如果图 G 中的一个路径包括每条边恰好一次，则该路径称为欧拉路径。如果一个回路是欧拉路径，则称为欧拉回路。具有欧拉回路的图称为欧拉图，具有欧拉路径但不具有欧拉回路的图称为半欧拉图。
- 定理一，存在欧拉路的条件：图是连通的，有且只有2个奇点。
- 定理二，存在欧拉回路的条件：图是连通的，有0个奇点。

  显然，既然每条边都要经过一次，对于欧拉路，除了起点和终点外，其余的每个点必然会进去一次，出去一次，所以除了起点和终点是奇点外，其余都是偶点。对于欧拉回路，每个点都要进去一次出去一次，显然都是偶点。

3.2 打印欧拉路径

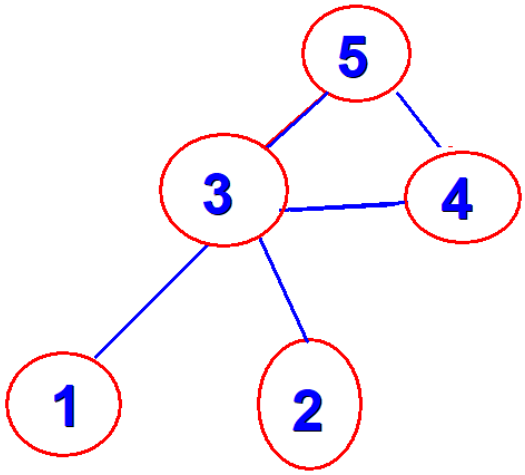
- 我们常说的一笔画问题，其实就是欧拉路径，如果还能回到起点，那么它就是欧拉回路。

```
一笔画问题    一本通p1341

时间限制:1000ms          内存限制:65536KB
【题目描述】
如果一个图存在一笔画，则一笔画的路径叫做欧拉路，如果最后又回到起点，那这个路径叫做欧拉回路。
根据一笔画的两个定理，如果寻找欧拉回路，对任意一个点执行深度优先遍历；找欧拉路，则对一个奇点执行dfs，时间复杂度为O(m+n)，m为边数，n是点数。

【输入】
第一行n，m，有n个点，m条边，以下m行描述每条边连接的两点， n < 50

【输出】
欧拉路或欧拉回路,输出一条路径即可。

【输入样例】
5 5
1 2
2 3
3 4
4 5
5 1
【输出样例】
1 5 4 3 2 1
```

求一笔画问题，使用深度优先遍历即可。如果寻找欧拉回路，对任意一个点执行深度优先遍历都可以，如果要找欧拉路，则要对一个奇点执行dfs, 时间复杂度为O(m+n), m为边数，n为顶点数。

```
#include <bits/stdc++.h>
#define ll long long
using namespace std;
const int maxn=1010;
int degree[maxn],path[maxn];
```

```cpp
int g[1010][1010], cnt, n, m;
template<class T> inline T read(T &x){
    x = 0; T f=1; char ch=getchar();
    while(!isdigit(ch)){if(ch=='-')f=-1;ch=getchar();}
    while(isdigit(ch)){x=(x<<3)+(x<<1)+(ch^48);ch=getchar();}
    x*=f; return (T)1;
}
template<class T> inline T read(T &x, T &y){return read(x)&&read(y);}
void dfs(int u){
    for(int i=1;i<=n;i++){
        if (g[u][i]){
            g[u][i]=g[i][u]=0;
            dfs(i);
        }
    }
    path[++cnt] = u;
}
int main()
{
    ios::sync_with_stdio(false);cin.tie(0);
    cin >> n >> m;
    int u, v;
    for(int i=1;i<=m;i++){
        cin >> u >> v;
        g[u][v]=g[v][u] = 1;
        degree[u]++; degree[v]++;
    }
    int start = 1;
    for(int i=1;i<=n;i++){
        if (degree[i]%2==1) {
            start = i;
        }
    }
    dfs(start);
    for(int i=1;i<=cnt;i++) printf("%d ", path[i]);
    return 0;
}
```



先序存储路径： 1->3->2->4->5->3

后序存储路径: 2->3->5->4->3->1 可见后序存储路径为正确的欧拉路

3.3 是否存在欧拉路径问题

- 是否存在欧拉路问题
-

```
一笔画问题(NYOJ42)
时间限制:1s      空间限制:125mb
描述
zyc从小就比较喜欢玩一些小游戏，其中就包括画一笔画，他想请你帮他写一个程序，判断一个图是否能够用一笔画下来。
规定，所有的边都只能画一次，不能重复画。

输入
第一行只有一个正整数N(N<=10)表示测试数据的组数。 每组测试数据的第一行有两个正整数P,Q(P<=1000,Q<=2000)，分别表示这个画中有多少个顶点和多少条连线。（点的编号从1到P） 随后的Q行，每行有两个正整数A,B(0 < A,B < P)，表示编号为A和B的两点之间有连线。

输出
如果存在符合条件的连线，则输出"Yes", 如果不存在符合条件的连线，输出"No"。

样例输入
2
4 3
1 2
1 3
1 4
4 5
1 2
2 3
1 3
1 4
3 4
样例输出
No
Yes
```

```cpp
#include <bits/stdc++.h>
#define ll long long
using namespace std;
const int maxn=1010;
int p, q, n, cnt, sum;
vector<int> G[maxn];
bool vis[maxn];
void dfs(int u){
    vis[u] = true;
    cnt ++;
    if (G[u].size()%2==1) sum++;
    for(int i=0;i<G[u].size();i++){
        int v = G[u][i];
        if (!vis[v]){
            dfs(v);
        }
    }
```

```
}
int main()
{
    ios::sync_with_stdio(false);cin.tie(0);
    cin >> n;
    while(n--){
        cin >> p >> q;
        memset(vis,0,sizeof(vis));
        for(int i=1;i<=p;i++) G[i].clear();
        cnt = 0;
        sum = 0;
        int u, v;
        for(int i=1;i<=q;i++){
            cin >> u >> v;
            G[u].push_back(v);
            G[v].push_back(u);
        }
        dfs(1);
        if (cnt==p&&(sum==0||sum==2)){
            cout<<"Yes"<<endl;
        }
        else cout<<"No"<<endl;
    }
    return 0;
}
```

4. 作业

4.1 骑马修栅栏 https://www.luogu.com.cn/problem/P2731

-

```
时间限制: 1000ms          内存限制:125mb
【题目描述】
农民John每年有很多栅栏要修理。他总是骑着马穿过每一个栅栏并修复它破损的地方。
John是一个与其他农民一样懒的人。他讨厌骑马，因此从来不两次经过一个一个栅栏。你必须编一个程序，读入栅栏网络的描述，并计算出一条修栅栏的路径，使每个栅栏都恰好被经过一次。John能从任何一个顶点(即两个栅栏的交点)开始骑马，在任意一个顶点结束。

每一个栅栏连接两个顶点，顶点用1到500标号(虽然有的农场并没有500个顶点)。一个顶点上可连接任意多(≥1)个栅栏。所有栅栏都是连通的(也就是你可以从任意一个栅栏到达另外的所有栅栏)。

你的程序必须输出骑马的路径(用路上依次经过的顶点号码表示)。我们如果把输出的路径看成是一个500进制的数，那么当存在多组解的情况下，输出500进制表示法中最小的一个(也就是输出第一个数较小的，如果还有多组解，输出第二个数较小的，等等)。 输入数据保证至少有一个解。

【输入】
第1行:一个整数F(1≤F≤1024)，表示栅栏的数目;
第2到F+1行:每行两个整数i,j(1≤=i,j≤500)表示这条栅栏连接i与j号顶点。

【输出】
输出应当有F+1行，每行一个整数，依次表示路径经过的顶点号。注意数据可能有多组解，但是只有上面题目要求的那一组解是认为正确的。

```

```
【输入样例】
9
1 2
2 3
3 4
4 2
4 5
2 5
5 6
5 7
4 6
【输出样例】
1
2
3
4
2
5
4
6
5
7
```

4.2 欧拉路径 https://www.acwing.com/problem/content/1621/

-

```
时间限制:0.3s        空间限制:64mb
在图论中，欧拉路径是图中的一条路径，该路径满足恰好访问每个边一次。
而欧拉回路是一条在同一顶点处开始和结束的欧拉路径。
它们最早由欧拉于 1736 年解决著名的哥尼斯堡七桥问题时提出。
事实证明，如果一个连通图的所有顶点的度数都为偶数，那么这个连通图具有欧拉回路，且这个图被称为欧拉图。
如果一个连通图中有两个顶点的度数为奇数，其他顶点的度数为偶数，那么所有欧拉路径都从其中一个度数为奇数的顶点开始，并在另一个度数为奇数的顶点结束。
具有欧拉路径但不具有欧拉回路的图被称为半欧拉图。
现在，给定一个无向图，请你判断它是欧拉图、半欧拉图还是非欧拉图。

输入格式
第一行包含两个整数 N 和 M，表示无向图的点和边的数量。
接下来 M 行，每行包含两个整数 a,b，表示点 a 和 b 之间存在一条边。
所有点的编号从 1~N。

输出格式
首先，在第一行按顺序输出点 1~N 中每个点的度数。
第二行输出对该图的判断，Eulerian（欧拉图），Semi-Eulerian（半欧拉图），Non-Eulerian（非欧拉图）。行尾不得有多余空格。

数据范围
1≤N≤500,1≤M≤N*(N-1)/2
输入样例1:
7 12
5 7
1 2
1 3
2 3
2 4
```

```
3 4
5 2
7 6
6 3
4 5
6 4
5 6
输出样例1:
2 4 4 4 4 4 2
Eulerian
输入样例2:
6 10
1 2
1 3
2 3
2 4
3 4
5 2
6 3
4 5
6 4
5 6
输出样例2:
2 4 4 4 3 3
Semi-Eulerian
输入样例3:
5 8
1 2
2 5
5 4
4 1
1 3
3 2
3 4
5 3
输出样例3:
3 3 4 3 3
Non-Eulerian
```